

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Distributed Computing

Michael Kosyakov
Associate Professor

Denis Tarakanov
Assistant Lecturer

ifmo.distributedclass.bot@gmail.com



Model of a distributed system

Model of a distributed system



All models are wrong, but some are useful

- RAM model vs Concurrency model
- Share everything vs Share nothing

Model of a distributed system



- A distributed system consists of a finite set of processes $\{P_1, P_2, \ldots P_N\}$ and a finite set of one-directional channels $\{C_{ij}\}$
  - Channel C_{ij} exists if process P_i can directly send messages to process P_j
  - Channels are assumed to have infinite buffers and to be error-free, and we do not make any assumptions on the ordering of messages
  - Processes, channels and communication are asynchronous
- Hence, distributed system is described by a directed graph in which the vertices represent processes and the edges represent channels

Model of a distributed system



- The state of a channel is the set of messages sent along the channel, but no yet received by receiving process
- The state of a process is characterized by the state of its local memory including the program counter and depends on the context
- The model of a process is defined by a set of states, an initial state (from this set), and a set of events
- An event *e* in a process *P* is an atomic action that may change the state of *P* itself and the state of *at most one* channel *C* incident on *P*

Events

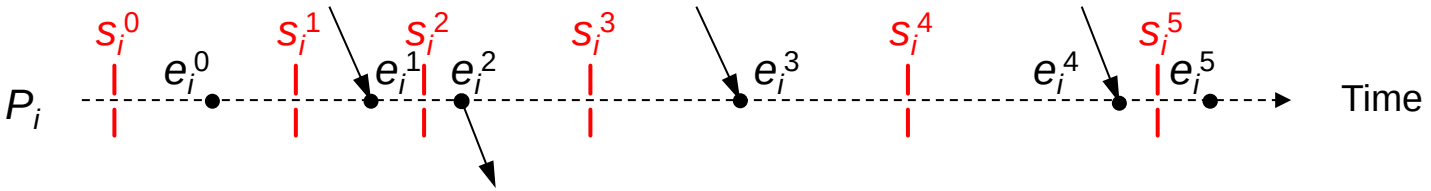
- An event *e* is defined by
  - 1) the process *P* in which the event occurs
  - 2) the state *s* of *P* immediately before the event
  - 3) the state *s'* of *P* immediately after the event
  - 4) the channel *C* (if any) whose state is altered by the event
  - 5) the message *m* (if any) sent along *C* (if *C* is a channel directed away from *P*) or received along *C* (if *C* is a channel directed towards *P*)
- Hence, we define *e* by the 5-tuple (*P*, *s*, *s'*, *m*, *C*), where *m* and *C* are equal to *null*, if the occurrence of *e* does not change the state of any channel

Events

- All events can be classified into three types
  - Internal event: changes the state of the process at which it occurs
  - Send event: changes the state of the process that sends the message and the state of the channel on which the message is sent
  - Receive event: changes the state of the process that receives the message and the state of the channel on which the message is received
- The events at a process are linearly ordered by their order of occurrence
  - Notation: e_i^x denotes the x-th event at process P_i

Process execution

- Lets denote e_i (s) = s' the state of the process P_i immediately after the occurrence of event e_i in state s
- Execution (run) of process P_i is a sequence of events e_i^0, e_i^1, … e_i^x, e_i^{x+1}, … where $s_i^{x+1} = e_i^x(s_i^x)$ and s_i^0 is the initial state of P_i



- Notation: $e_i \rightarrow_i e_i'$ if e_i comes before e_i' in P_i

Execution of distributed system

- The execution of a distributed system results in a set of events E occurred in the processes, i.e. $E = \cup_i E_i$ where E_i is the set of events occurred in process P_i
- A global state S of a distributed system is a set of process and channel states
- The initial global state is one in which the state of each process is its initial state and the state of each channel is the empty set
- The occurrence of an event changes the global state

Execution of distributed system



- Let denote e_i (S) the global state immediately after the occurrence of event e_i of process P_i in global state S
- The value of e_i (S) is defined only if event e_i *can occur* in global state S
- An internal event $e_i = (P_i, s, s', null, null)$ or send event $e_i = (P_i, s, s', m, C_{ij})$ can occur in global state S, if the state of process P_i in global state S is s
- A receive event $e_i = (P_i, s, s', m, C_{ji})$ can occur in global state S, if the state of process P_i in global state S is s and message m is in the channel C_{ji} and can be received from it

Execution of distributed system



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Global state e_i (S) immediately after the occurrence of event e_i is identical to S except that:
- In case of internal event $e_i = (P_i, s, s', null, null)$
  - the state of the process P_i in global state e (S) is s'
- In case of send event $e_i = (P_i, s, s', m, C_{ij})$
  - the state of the process P_i in global state e (S) is s' and the state of the channel C_{ij} is the same as in S with the message m added to the channel
- In case of receive event $e_i = (P_i, s, s', m, C_{ji})$
  - the state of the process P_i in global state e (S) is s' and the state of the channel C_{ji} is the same as in S with the message m removed from the channel

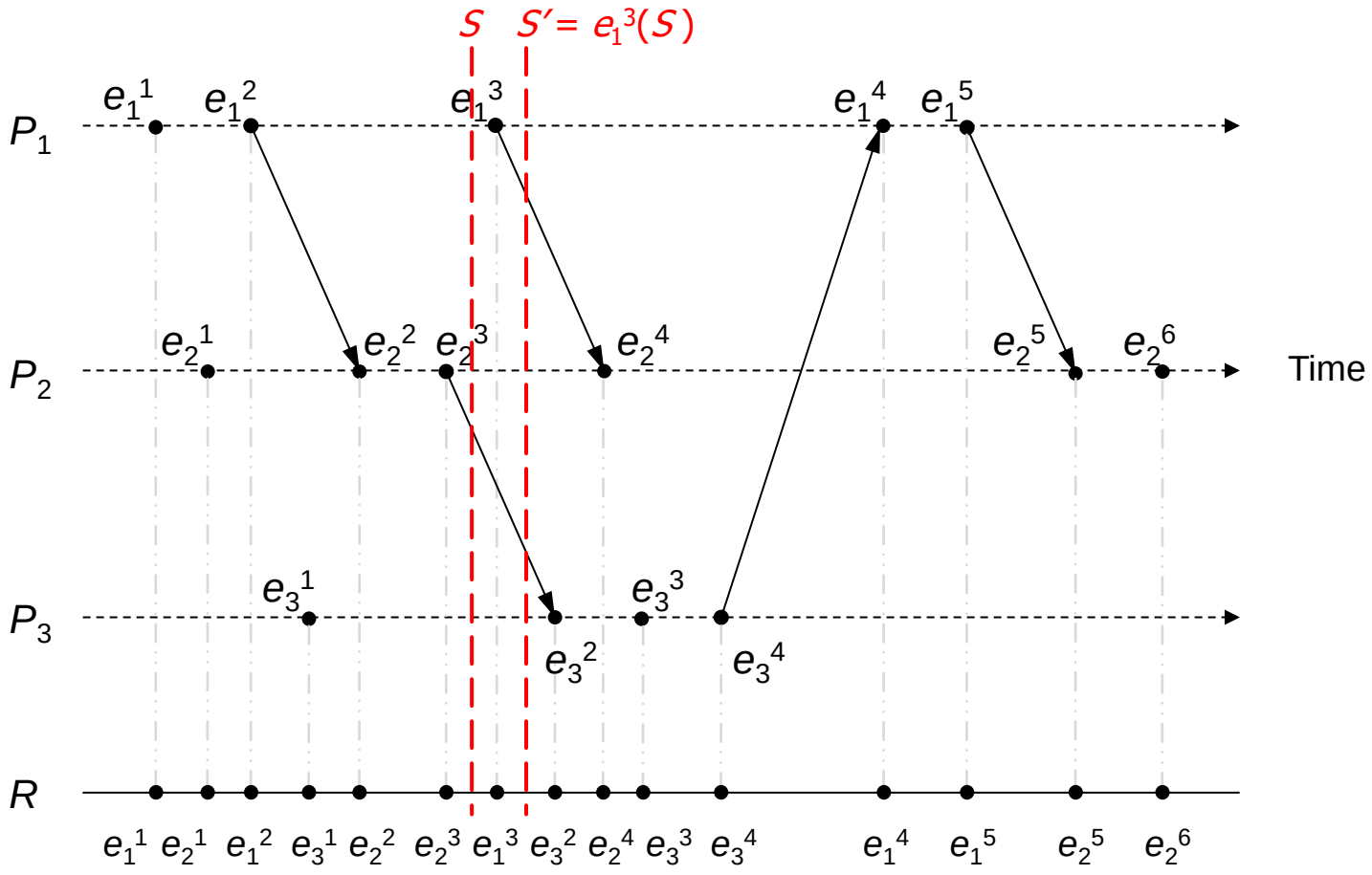
Execution of distributed system



- Let $R = (e^0, e^1, \ldots e^x, e^{x+1}, \ldots)$ be a global sequence of events occurred in processes of a distributed system, i.e. $e^x \in E$ for every $x \geq 0$
- We say that R is an execution (run) of the distributed system if and only if event e^x can occur in global state S^x, and $S^{x+1} = e^x(S^x)$ for every $x \geq 0$, and where S^0 is the initial global state
- Thus, all events in an execution are interleaved as if the whole system is running on a single processor
- Global state S^* is called reachable in execution R if there exists $k > 0$, so that $e^k(S^k) = S^*$ or $S^* = S^0$

Example of distributed execution

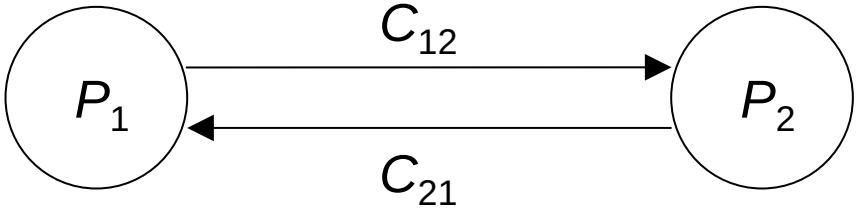
Space-time diagram



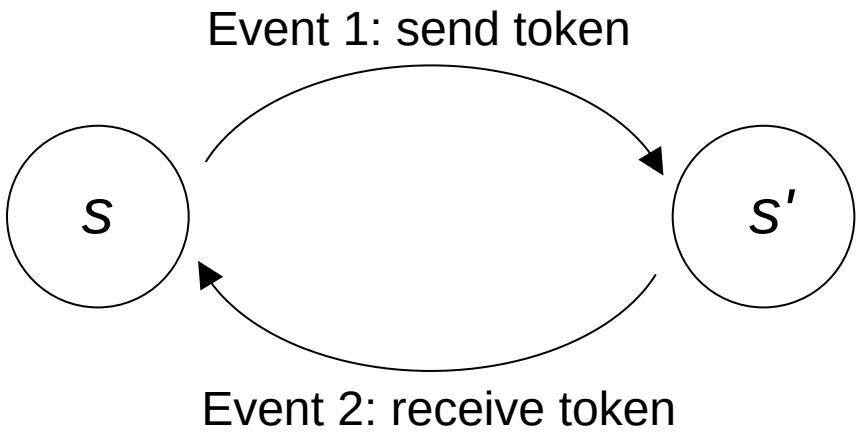
Single-token conservation example



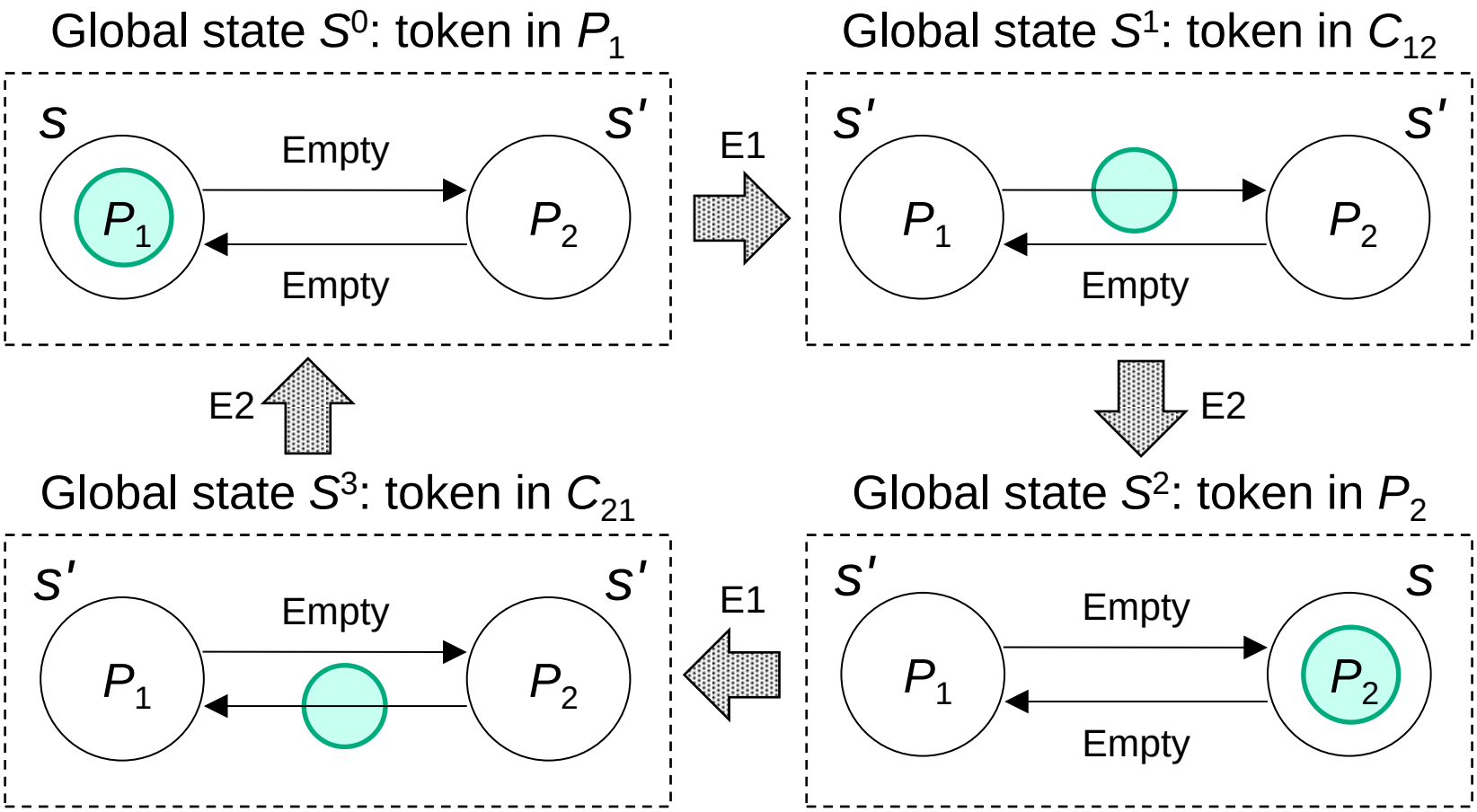
An example of topology of a distributed system:



State-transition diagram of a process (P_1 or P_2):



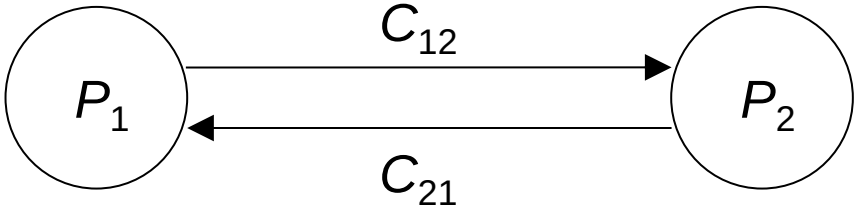
Global states and transitions



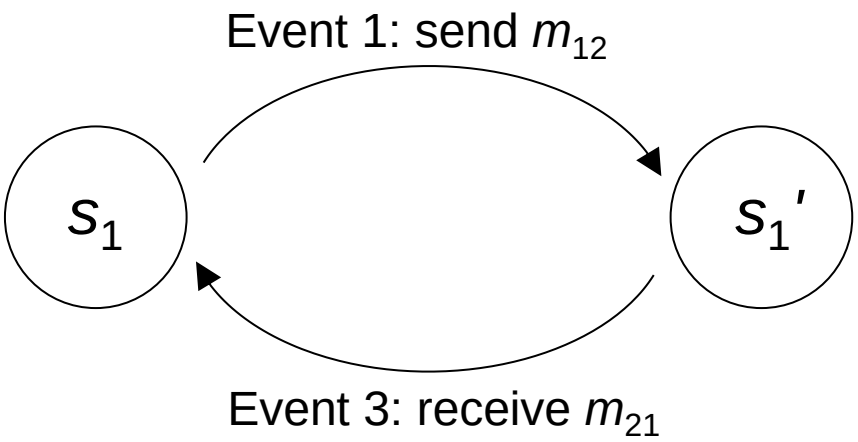
There is *exactly one* event possible in each global state

Example of nondeterministic run

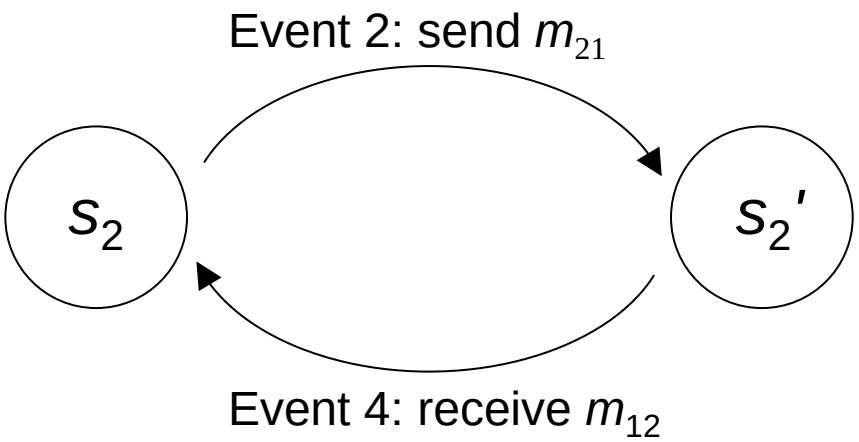
An example of topology of a distributed system:



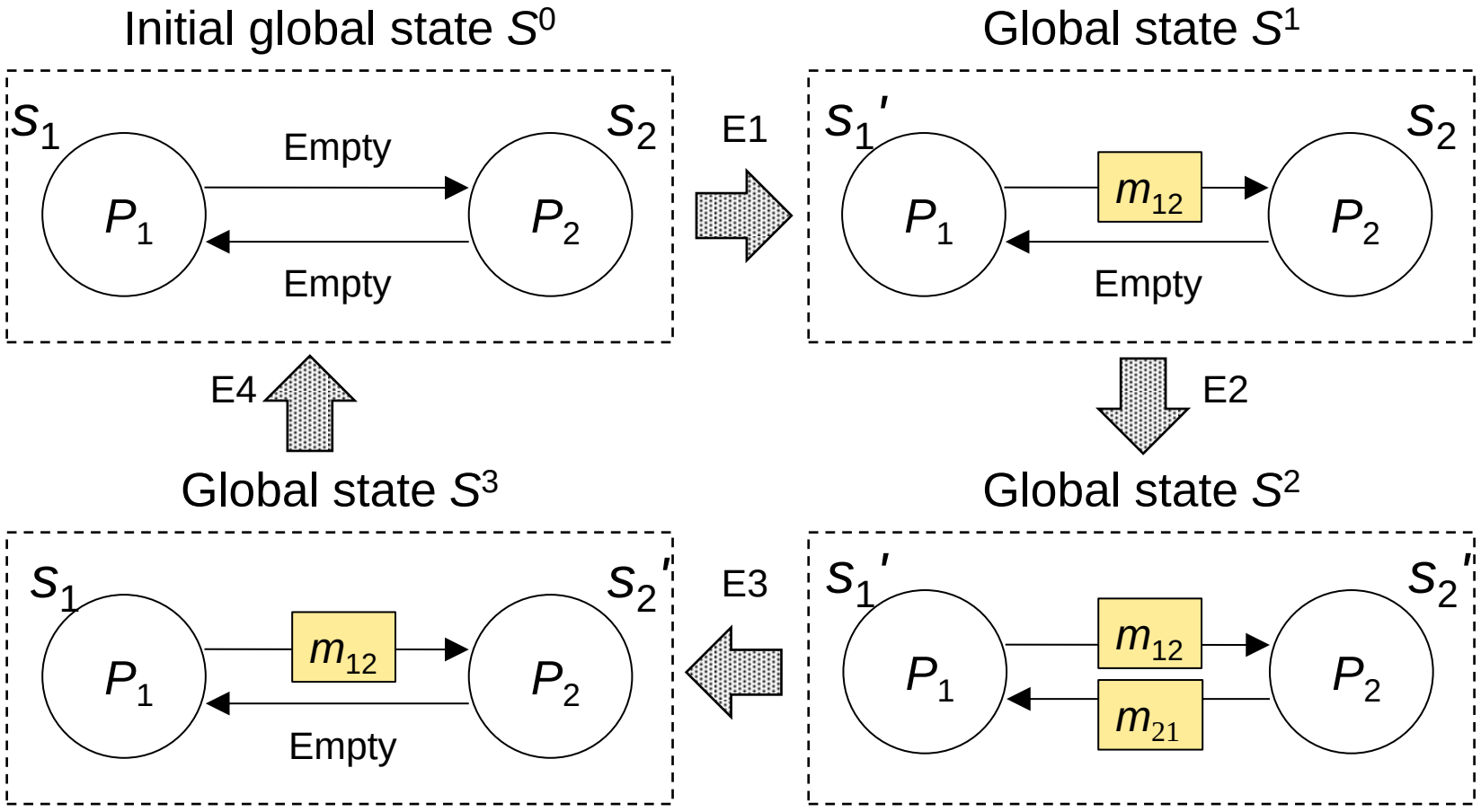
State-transition diagram for process P_1



State-transition diagram for process P_2

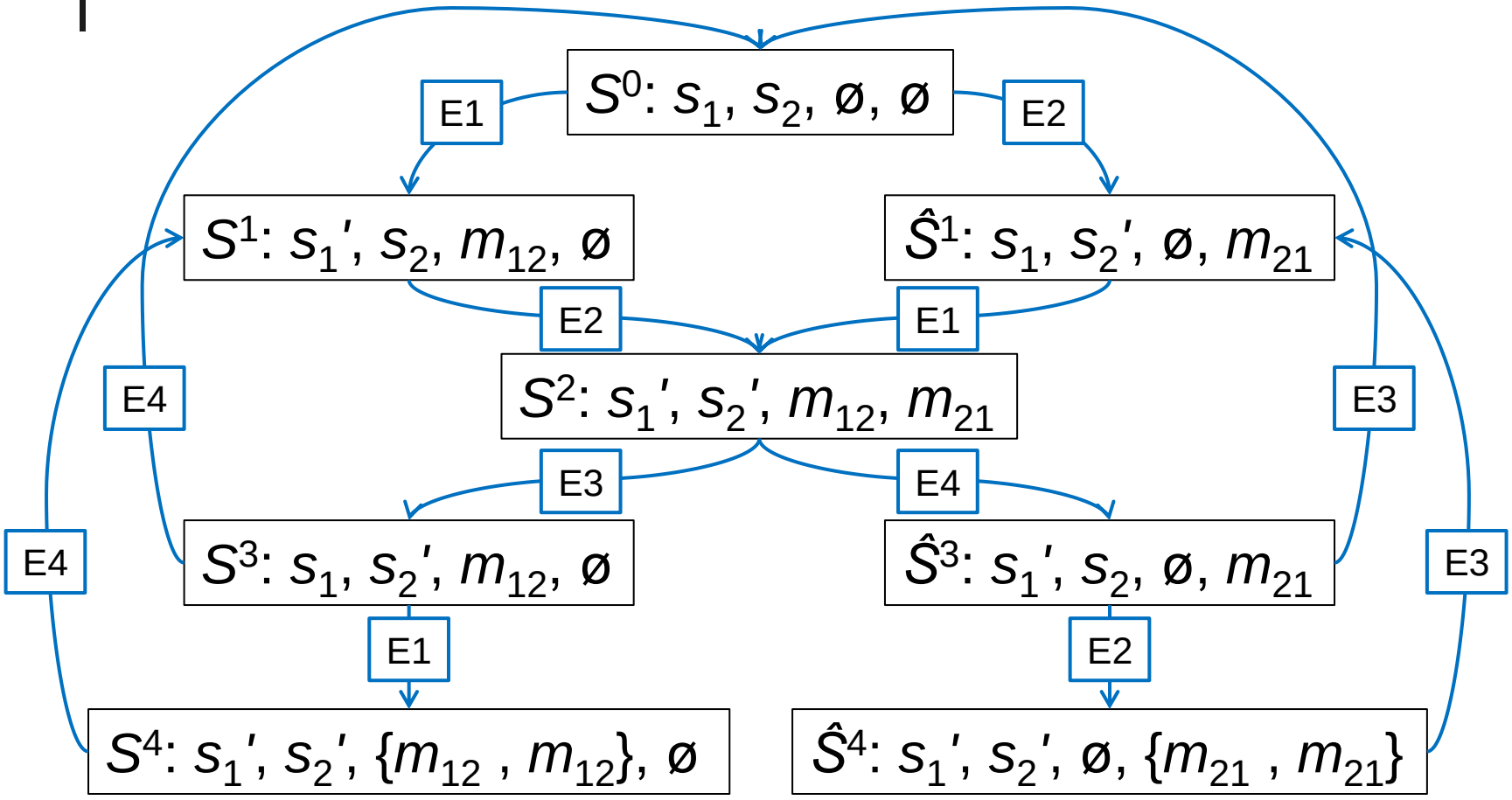


Example of nondeterministic run



There may be more than one transition allowable from a global state, i.e. execution of the distributed system may be different

Global states and transitions



A different sequence of events may produce a different sequence of *different* global states, e.g. execution E2, E1, E4, E3 goes through global states $\hat{S}^1$ and $\hat{S}^3$ which are not reachable in execution E1, E2, E3, E4

Execution of distributed system



while (*S* = non-terminal global state)

{

select nondeterministically an event *e*,
that can occur in global state *S*

execute the event *e* and put *S* = *e*(*S*)

}

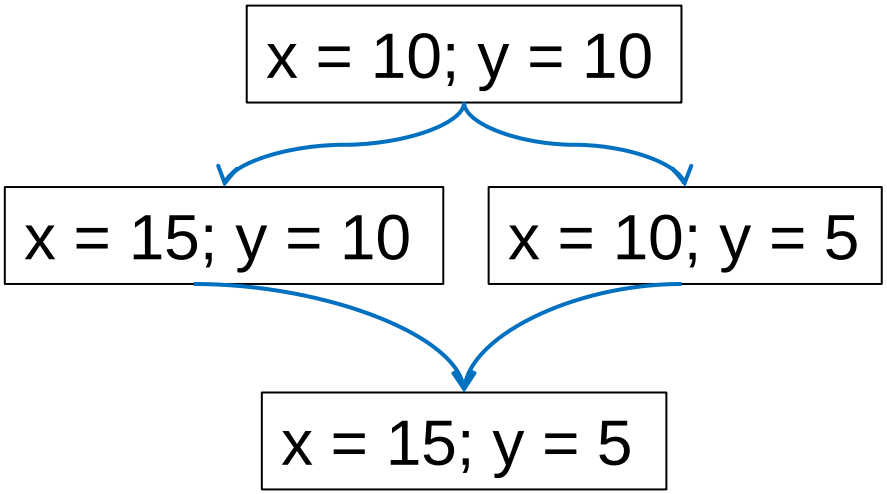
A system execution corresponds to a path (sequence of transitions) in the global-state-transition diagram starting at the initial global state

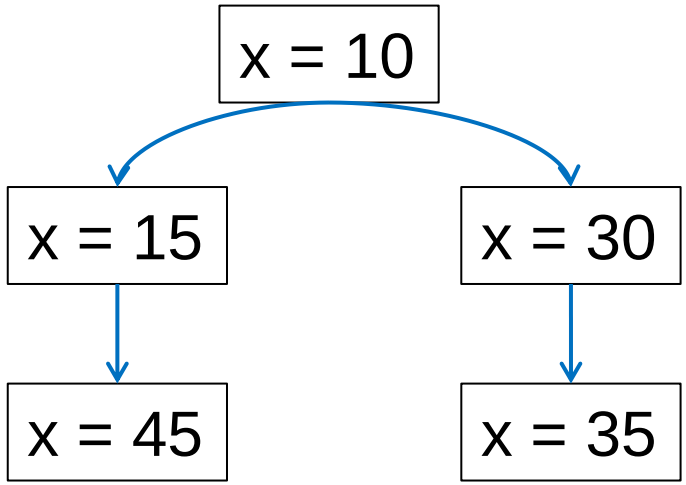
Nondeterminism may refer to ...



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Interleaving of independent actions on local variables	Resolution of a contention between critical actions on shared variable
x = x + 5; y = y – 5	x = x + 5; x = 3x





Source code with critical section



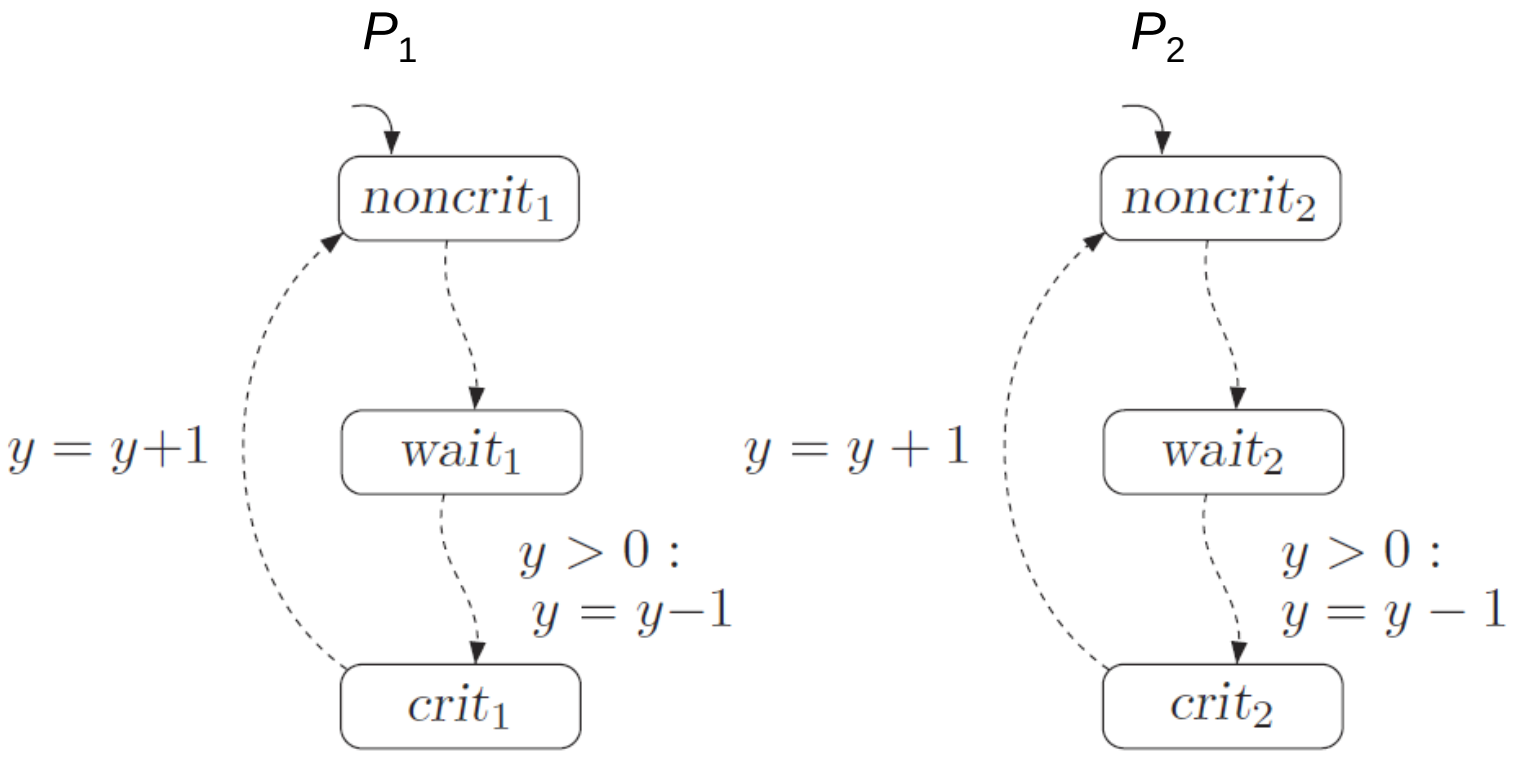
HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

P_1	P_2

while(true) {
…
/* noncritical actions */
…
request_cs();
/* critical actions */
release_cs();
…
/* noncritical actions */
…
}
	while(true) {
	…
	/* noncritical actions */
	…
	request_cs();
	/* critical actions */
	release_cs();
	…
	/* noncritical actions */
	…
	}

Mutual exclusion with shared variable

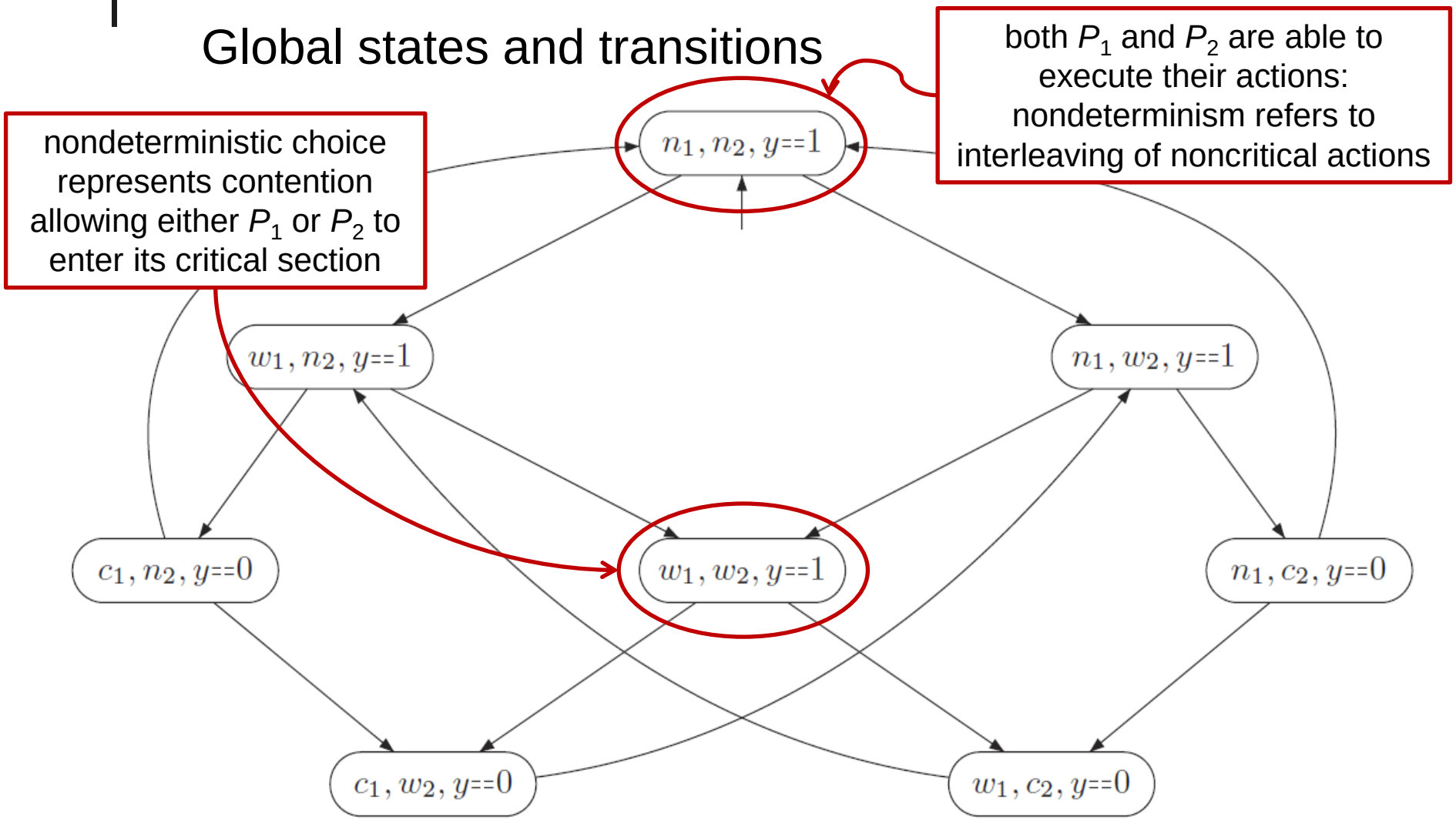
State-transition diagram for processes P_1 and P_2



When y = 1, the critical sections is free;
y = 0 indicates that the critical section is currently occupied by another process

Mutual exclusion with shared variable

Global states and transitions



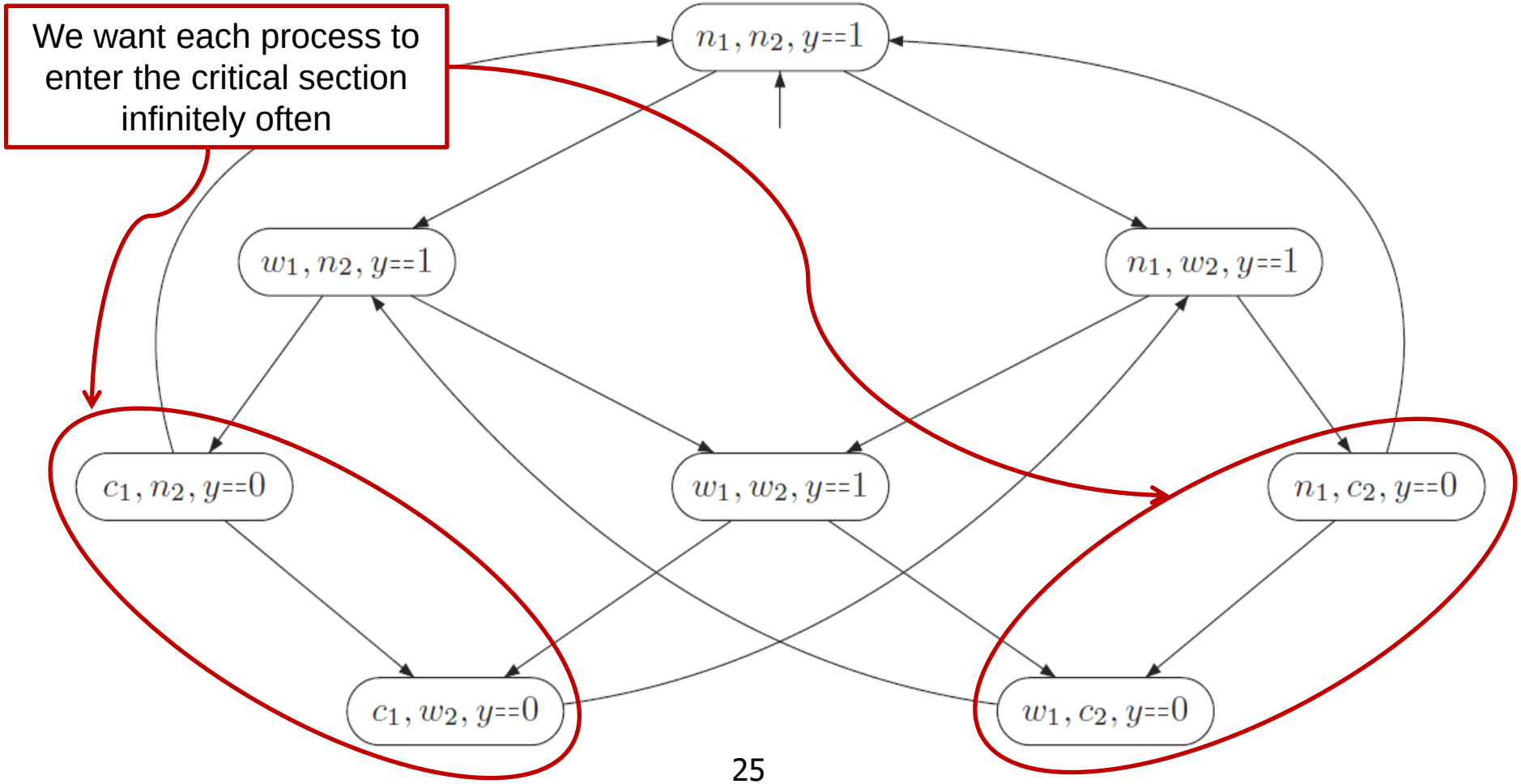
Christel Baier, Joost-Pieter Katoen. Principles of Model Checking, The MIT Press, 2008

Fair executions

- We need an “adequate” resolution of the nondeterministic decisions
- Fairness constraints are used to rule out executions that are considered to be unreasonable for the system under consideration
- Namely, to rule out executions where events *can* always (or infinitely often) occur, but actually *never* occur as a transition because the executions continues with other events
- An execution is *weakly* fair if no event can occur in infinitely many consecutive global states without occurring in the execution
- An execution is *strongly* fair if no event can occur in infinitely many global states without occurring in the execution

Mutual exclusion with shared variable

Global states and transitions

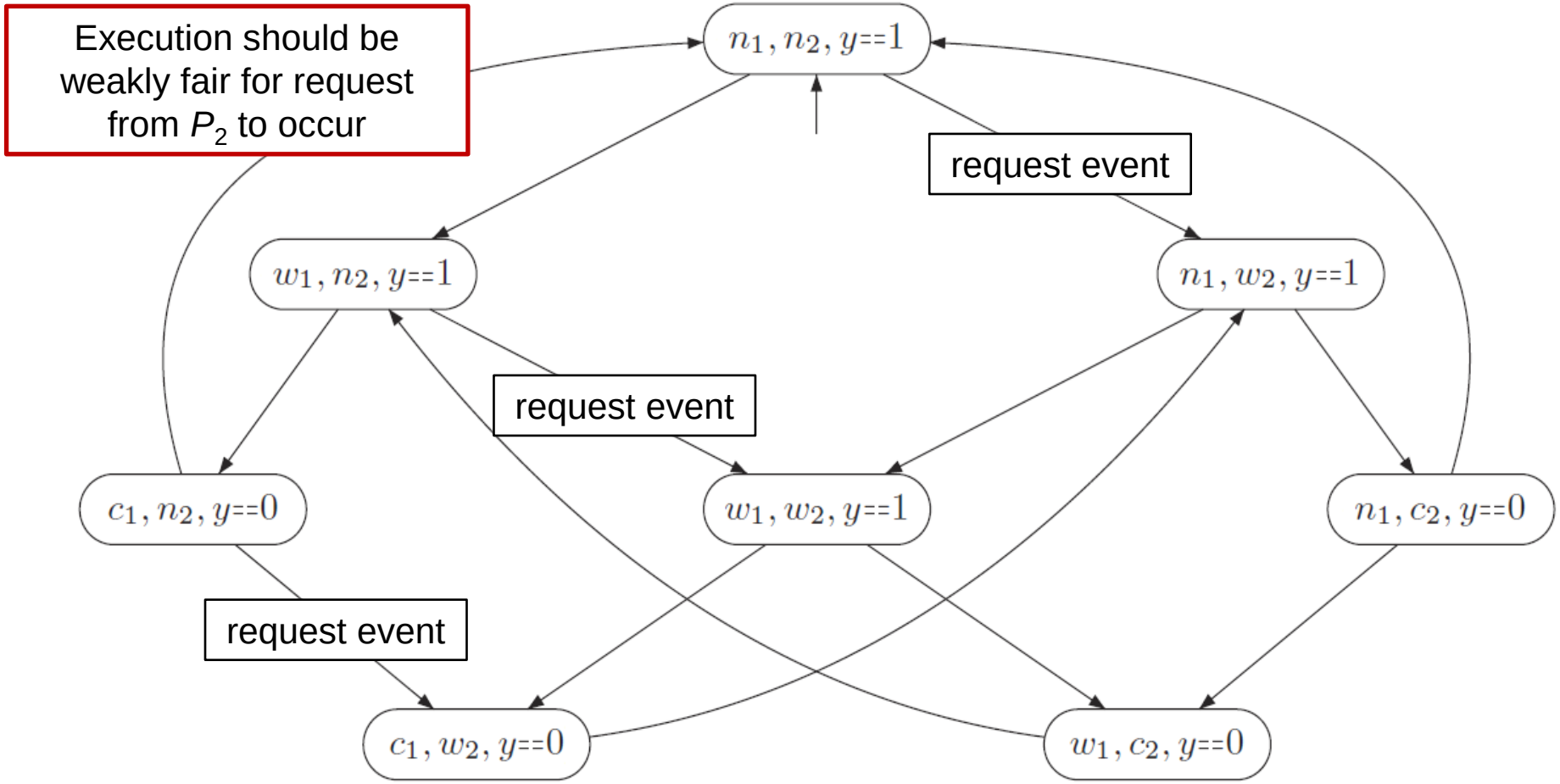


Christel Baier, Joost-Pieter Katoen. Principles of Model Checking, The MIT Press, 2008

Mutual exclusion with shared variable

Global states and transitions

Execution should be weakly fair for request from P_2 to occur

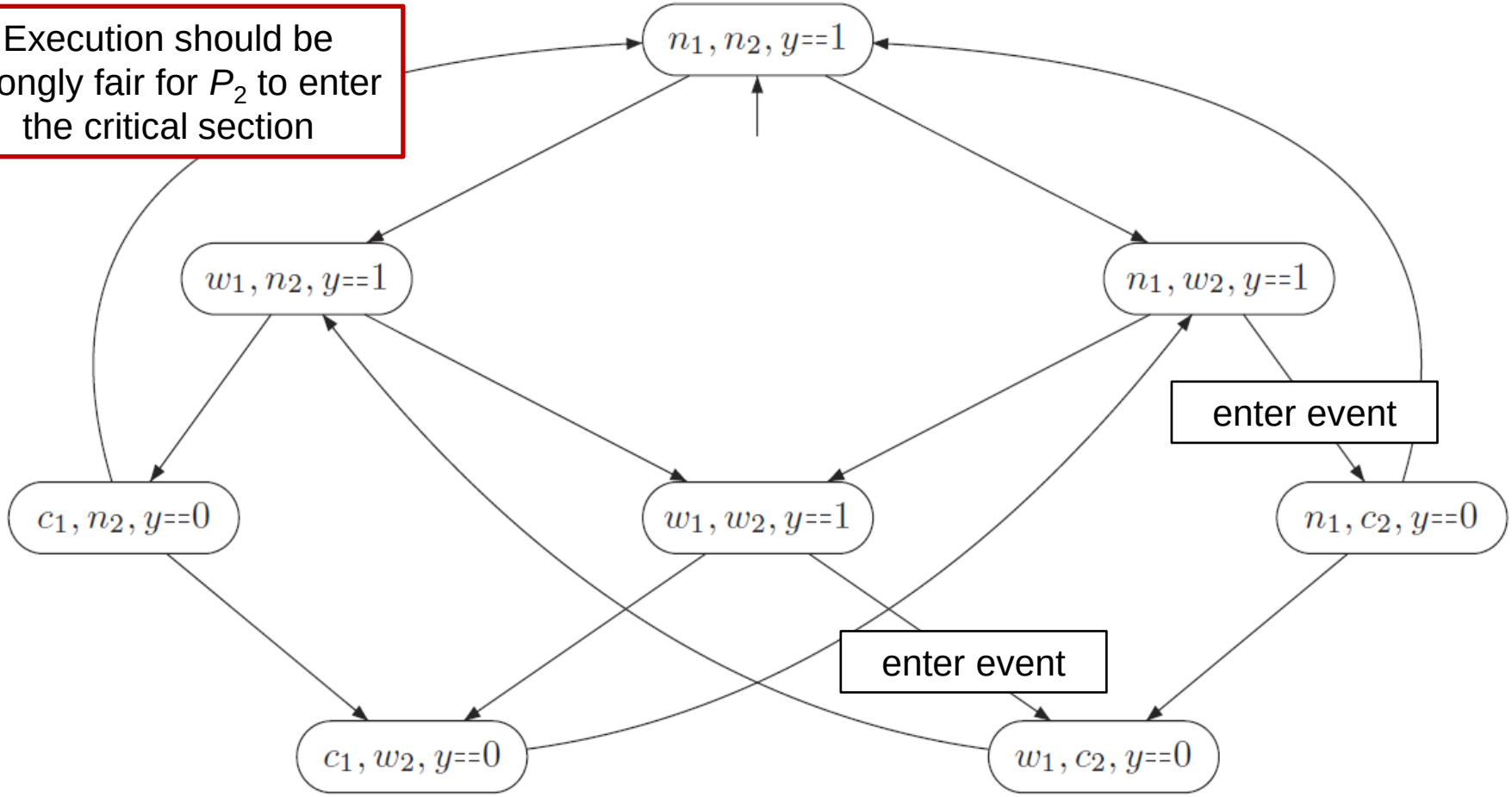


Christel Baier, Joost-Pieter Katoen. Principles of Model Checking, The MIT Press, 2008

Mutual exclusion with shared variable

Global states and transitions

Execution should be strongly fair for P_2 to enter the critical section



Christel Baier, Joost-Pieter Katoen. Principles of Model Checking, The MIT Press, 2008



MPI (non) fairness

MPI: A Message-Passing Interface Standard, v3.1

MPI makes no guarantee of *fairness* in the handling of communication.

Suppose that a send is posted. Then it is possible that the destination process repeatedly posts a receive that matches this send, yet the message is never received, because it is each time overtaken by another message, sent from another source.

Similarly, suppose that a receive was posted by a multi-threaded process. Then it is possible that messages that match this receive are repeatedly received, yet the receive is never satisfied, because it is overtaken by other receives posted at this node (by other executing threads). It is the programmer's responsibility to prevent starvation in such situations.

https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf

Proving properties of distributed algorithms



- When developing a distributed algorithm that solves a particular problem, it is necessary to show that the resulting algorithm really represents a correct solution to the problem
- There are two fundamental types of properties that are checked to confirm the correctness of a distributed algorithm:
- “Safety” properties, and
- “Liveness” properties

Safety properties

- Informally, the algorithm satisfies the safety property if "something bad never happens“
- A safety requirement imposes that a certain property must hold for *each* execution of the system in *every* global state reached in that execution
- E.g. for traffic light control system it means that any distribution of traffic signals should prohibit driving on intersecting directions at a crossroad at the same time

Liveness properties

- However, just because nothing bad can happen does not mean that anything good can happen
- Systems which do nothing always satisfy safety properties as just nothing happens and, hence, nothing bad happens as well
- E.g. for traffic light control example, safety property could be satisfied by permission to drive in one direction only
- So safety properties by themselves are not enough… they should be complemented by properties that require some progress
- Such properties are called “liveness” properties (or sometimes “progress” properties)

Liveness properties

- Informally, the algorithm satisfies the liveness property if "something good eventually happens“
- A liveness requirement imposes that a certain property must hold for *each* execution of the system in *some* global state reached in that execution
- E.g. for traffic light control system this property should be expressed in ensuring that eventually it is possible to drive in any direction at a crossroad